

Module-8 Python – Advanced Python Programming

1. PRINTING ON SCREEN

THEORY :

1. Introduction to the print() function in Python.



The print() function in Python is used to display output on the screen. It is one of the most commonly used functions and helps in showing text, variables, and results of operations.

Example:

```
print("Hello, World!")
```

2. Formatting outputs using f-strings and format().



1. Using f-strings:

F-strings (formatted string literals) allow embedding variables directly inside strings using {}. It is simple and readable.

Example:

```
name = "Kaif"
age = 22
print(f"My name is {name} and I am {age} years old.")\
```

2. Using format() method:

The format() method allows inserting values into placeholders {} in a string.

Example:

```
name = "Kaif"
age = 22
print("My name is {} and I am {} years old.".format(name, age))
```

2. READING DATA FROM KEYBOARD

THEORY :

1. Using the input() function to read user input from the keyboard.



The input() function in Python is used to take input from the user through the keyboard. It always returns the entered value as a string.

Example:

```
name = input("Enter your name: ")  
print("Hello", name)
```

2. Converting user input into different data types (e.g., int, float, etc.).



Since input() returns data as a string, we need to convert it into required data types using type casting functions like int(), float(), etc.

Examples:

```
age = int(input("Enter your age: "))    # Convert to integer  
price = float(input("Enter price: "))  # Convert to float  
  
print(age)  
print(price)
```

3. OPENING AND CLOSING FILES

THEORY :

1. Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').



Python provides different modes to open files depending on the operation:

- 'r' → Read mode (default). Opens file for reading. Error if file does not exist.
- 'w' → Write mode. Creates a new file or overwrites existing content.
- 'a' → Append mode. Adds data to the end of the file without deleting existing content.
- 'r+' → Read and write mode. File must exist.
- 'w+' → Write and read mode. Creates new file or overwrites existing one.

Example: `file = open("example.txt", "r")`

2. Using the open() function to create and access files.



The open() function is used to create a file or open an existing file. It takes two main arguments: filename and mode.

Syntax: `file = open("filename", "mode")`

Example: `file = open("data.txt", "w")`

3. Closing files using close().



The close() function is used to close the file after completing operations. It frees system resources and ensures data is saved properly.

Example: `file.close()`

4. READING AND WRITING FILES

THEORY :

1. Reading from a file using read(), readline(), readlines().



Python provides three main methods to read data from a file:

- read() → Reads the entire file content as a single string.
- readline() → Reads one line at a time.
- readlines() → Reads all lines and returns them as a list.

Example:

```
file = open("sample.txt", "r")

print(file.read())    # Read full content
file.seek(0)
print(file.readline()) # Read one line
file.seek(0)
print(file.readlines()) # Read all lines as list
file.close()
```

2. Writing to a file using write() and writelines().



Python provides methods to write data into files:

- write() → Writes a single string to the file.
- writelines() → Writes multiple strings (list of strings) to the file.

Example:

```
file = open("sample.txt", "w")

file.write("Hello\n")
file.writelines(["Line 1\n", "Line 2\n"])

file.close()
```

5. EXCEPTION HANDLING

THEORY :

1. Introduction to exceptions and how to handle them using try, except, and finally.



Exceptions are errors that occur during the execution of a program. Python provides try, except, and finally blocks to handle these errors gracefully without stopping the program.

- try → Code that may cause an error
- except → Handles the error
- finally → Always executes (whether error occurs or not)

Example:

```
try:
    num = int(input("Enter a number: "))
    print(num)
except:
    print("Invalid input!")
finally:
    print("Program ended.")
```

2. Understanding multiple exceptions and custom exceptions.



Multiple Exceptions:

Handling more than one type of error using multiple except blocks.

Custom Exceptions:

User-defined exceptions created using classes (inheriting from Exception).

Example:

Multiple exceptions

try:

 a = int(input("Enter number: "))

 b = int(input("Enter number: "))

 print(a / b)

except ZeroDivisionError:

 print("Cannot divide by zero")

except ValueError:

 print("Invalid input")

Custom exception

class MyError(Exception):

 pass

try:

 raise MyError("This is a custom exception")

except MyError as e:

 print(e)

6. CLASS AND OBJECTS (OOP CONCEPT)

THEORY :

1. Understanding the concepts of classes, objects, attributes, and methods in Python.



- **Class** → A blueprint or template used to create objects.
- **Object** → An instance of a class.
- **Attributes** → Variables that belong to a class or object.
- **Methods** → Functions defined inside a class.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name    # attribute
        self.age = age      # attribute

    def display(self):      # method
        print("Name:", self.name)
        print("Age:", self.age)

# Object creation
s1 = Student("Kaif", 22)
s1.display()
```

Example explanation:

- Student is a **class**
- s1 is an **object** of the class
- name and age are **attributes**
- display() is a **method**

2. Difference between local and global variables.



Local Variable:

A variable declared inside a function or method. It can only be used within that function.

Global Variable:

A variable declared outside all functions. It can be accessed anywhere in the program.

Example:

```
x = 10 # Global variable
```

```
def show():
```

```
    y = 5 # Local variable
```

```
    print("Local variable:", y)
```

```
    print("Global variable:", x)
```

```
show()
```

```
print("Global variable outside function:", x)
```

7. INHERITANCE

THEORY :

1. Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.



Inheritance is a concept where one class (**child / derived class**) gets the properties and methods of another class (**parent / base class**). It helps in **code reuse** and avoids repeating the same code.

1. Single Inheritance

One child class inherits from one parent class.

Example:

```
class Parent:
```

```
    def show(self):  
        print("This is parent class")
```

```
class Child(Parent):
```

```
    def display(self):  
        print("This is child class")
```

```
c = Child()
```

```
c.show()
```

```
c.display()
```

2. Multilevel Inheritance

A class inherits from another class, and then another class inherits from it (chain).

Example:

```
class Grandparent:
```

```
    def show1(self):  
        print("Grandparent")
```

```
class Parent(Grandparent):
```

```
    def show2(self):  
        print("Parent")
```

```
class Child(Parent):
```

```
    def show3(self):  
        print("Child")
```

```
c = Child()
```

```
c.show1()
```

```
c.show2()
```

```
c.show3()
```

3. Multiple Inheritance

One child class inherits from more than one parent class.

Example:

```
class Father:
```

```
    def show1(self):  
        print("Father")
```

```
class Mother:
```

```
    def show2(self):  
        print("Mother")
```

```
class Child(Father, Mother):
```

```
    def show3(self):  
        print("Child")
```

```
c = Child()
```

```
c.show1()
```

```
c.show2()
```

```
c.show3()
```

4. Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

Example:

```
class Parent:
```

```
    def show(self):
```

```
        print("Parent class")
```

```
class Child1(Parent):
```

```
    pass
```

```
class Child2(Parent):
```

```
    pass
```

```
c1 = Child1()
```

```
c2 = Child2()
```

```
c1.show()
```

```
c2.show()
```

5. Hybrid Inheritance

Combination of two or more types of inheritance (e.g., multiple + multilevel).

Example:

```
class A:
```

```
    def showA(self):  
        print("Class A")
```

```
class B(A):
```

```
    def showB(self):  
        print("Class B")
```

```
class C(A):
```

```
    def showC(self):  
        print("Class C")
```

```
class D(B, C):
```

```
    def showD(self):  
        print("Class D")
```

```
d = D()
```

```
d.showA()
```

```
d.showB()
```

```
d.showC()
```

```
d.showD()
```

2. Using the `super()` function to access properties of the parent class.



The `super()` function is used to call methods or constructors of the parent class from the child class. It helps in reusing code and avoiding direct reference to the parent class.

Example:

```
class Parent:
    def __init__(self):
        print("Parent constructor")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child constructor")

obj = Child()
```

8. METHOD OVERLOADING AND OVERRIDING

THEORY :

1. Method overloading: defining multiple methods with the same name but different parameters.



```
class Demo:
    def add(self, a, b=0, c=0):
        return a + b + c

obj = Demo()

print(obj.add(5, 10))    # 2 arguments
print(obj.add(5, 10, 15)) # 3 arguments
```

2. Method overriding: redefining a parent class method in the child class.



Method overriding occurs when a child class defines a method with the same name as a method in the parent class. The child class method overrides the parent class method.

Example:

```
class Parent:
    def show(self):
        print("This is parent method")

class Child(Parent):
    def show(self):
        print("This is child method")

obj = Child()
obj.show()
```

9. SQLite3 AND PyMySQL (DATABASE CONNECTORS)

THEORY :

1. Introduction to SQLite3 and PyMySQL for database connectivity



SQLite3

- SQLite3 is a lightweight, built-in database module in Python (sqlite3 comes pre-installed).
- It is a serverless database, meaning no need to install or run a separate database server.
- Data is stored in a single file on your system (e.g., data.db).
- It is best for small applications, testing, and local storage.

Key Features:

- No setup required (zero configuration)
- Fast and simple to use
- Supports standard SQL queries (CREATE, INSERT, SELECT, etc.)
- Good for desktop apps, small projects, or learning

Example:

```
import sqlite3
```

```
con = sqlite3.connect("data.db")
```

```
cursor = con.cursor()
```

```
cursor.execute("CREATE TABLE student(id INT, name TEXT)")
```

```
cursor.execute("INSERT INTO student VALUES(1, 'Kaif')")
```

```
con.commit()
```

```
con.close()
```

PyMySQL

- PyMySQL is a Python library used to connect to a MySQL database server.
- Unlike SQLite, it requires a separate MySQL server to be installed and running.
- It is used for large-scale applications and web development.

Key Features:

- Works with MySQL server
- Supports multi-user access
- Suitable for web apps and production systems
- Requires installation:

pip install pymysql

Example:

```
import pymysql

con = pymysql.connect(
    host="localhost",
    user="root",
    password="",
    database="test"
)

cursor = con.cursor()
cursor.execute("SELECT * FROM student")

for row in cursor:
    print(row)

con.close()
```

2. Creating and executing SQL queries from Python using these connectors.



Python allows you to interact with databases like **SQLite3** and **MySQL (using PyMySQL)** by writing and executing SQL queries directly in your code.

We use the following steps:

1. Connect to the database
2. Create a cursor object
3. Execute SQL queries using `execute()`
4. Fetch data using `fetchone()`, `fetchall()`
5. Close the connection

Example (SQLite3):

```
import sqlite3
```

```
conn = sqlite3.connect("example.db")  
cursor = conn.cursor()
```

```
cursor.execute("CREATE TABLE IF NOT EXISTS student(id INTEGER, name  
TEXT)")  
conn.commit()
```

```
conn.close()
```

10. SEARCH AND MATCH FUNCTIONS

THEORY :

1. Using `re.search()` and `re.match()` functions in Python's `re` module for pattern matching.



Python's `re` module is used for **regular expressions (pattern matching)**.

- `re.search()` → Searches the pattern anywhere in the string.
- `re.match()` → Checks for the pattern only at the beginning of the string.

Example:

```
import re
```

```
text = "Hello Python"
```

```
# search()
print(re.search("Python", text))
```

```
# match()
print(re.match("Hello", text))
```

2. Difference between search and match.



The difference between **`search()`** and **`match()`** in Python (from the `re` module) is mainly about **where the pattern is checked** in the string.

`re.match()`

- Checks for a match **only at the beginning** of the string.
- If the pattern is not at the start → returns `None`.

Example:

```
import re
```

```
text = "hello world"
```

```
print(re.match("hello", text)) #  Match
```

```
print(re.match("world", text)) #  None
```

re.search()

- Checks for a match **anywhere in the string**.
- Returns the first occurrence of the pattern.

Example:

```
import re
```

```
text = "hello world"
```

```
print(re.search("hello", text)) #  Match
```

```
print(re.search("world", text)) #  Match
```

Differences

Feature	<code>match()</code>	<code>search()</code>
Position	Beginning only	Anywhere in string
Flexibility	Less flexible	More flexible
Use case	Validate start of string	Find pattern in text